

Late Breaking Results: Source-Aware Adaptive Cache Management for CXL-enabled Disaggregated Memory Sharing

Qianyu Cheng[†], Jiajun Ji[†], Teng Wang, Zihan Wang, Lei Gong, Chao Wang, Xuehai Zhou

University of Science and Technology of China, Hefei 230026, China

{qycheng, jiajunji, wangzh196}@mail.ustc.edu.cn, {wangt635, leigong0203, cswang, xhzhou}@ustc.edu.cn

Abstract—Dynamic workloads running on multiple hosts will bring changing access patterns on CXL-enabled shared disaggregated memory. Existing works often un-traceably cache multi-source accesses, making it hard to exploit each host’s access behavior and assure service quality. Our solution *Alchemy* jointly optimizes cache replacement and bypassing and runs as an online reinforcement learning agent with source-aware adaptivity. It gives rewards derived from sampling-based action effectiveness and per-host macro performance. The multi-host prototype-based results on FPGAs show 8.71%-14.56% reduction in average access latency over LRU policy and 44x faster than the hardware-efficient ICGMM method in decision-making with comparable overhead.

Index Terms—Compute Express Link (CXL), Disaggregated Memory, Heterogeneity, Reinforcement Learning.

I. INTRODUCTION

Modern data-intensive applications such as in-memory databases, graph analytics, and large language models show the urgent need for tremendous memory capacity. However, the slow growth of memory scalability in data centers creates a performance bottleneck known as the *memory wall*. Researchers have explored solutions like memory tiering with non-volatile memory and memory disaggregation. Recent advances in general device interconnect standards have promoted Compute Express Link (CXL), which enables memory semantics for disaggregation. A CXL-based DM architecture consists of *compute nodes (CNs)*, *memory nodes (MNs)*, and an interconnect CXL fabric. *CNs* are host endpoints with limited local memory, while *MNs* are CXL devices with massive shared memory, including DRAM cache and low-cost, high-density SSDs.

However, existing CXL-enabled DM system [1] still faces challenges, especially for efficient cache management. The latency gap between local and remote CXL memory devices remains significant, and dynamic workload mixes introduce substantial access penalties in multi-*CN* systems. Fig. 1(a) highlights the dynamic nature of workloads. For workloads running on a system with 4 *CNs* and LRU policy, an average of 46% of cache blocks were not reused before eviction, with varying amounts of future reused blocks from 0.8% to 87% (gray segments of bars). It emerges the need to switch between multiple policies including cache bypassing and replacement-based policy. Furthermore, inter-*CN* interference exacerbates this issue by evicting critical blocks from the cache, causing QoS degradation. Fig. 1(b) illustrates the impact of introducing multiple *CNs* to a DM system with different workloads. Prior cache frameworks such as Glider [2] and ICGMM [3] are based on multiple policies, but these works only learn from the static workload behavior on clairvoyant policies (i.e., Belady’s OPT) or the distribution of existing workload traces, lacking long-term adaptability to dynamic workloads.

To address these challenges, This paper designed *Alchemy*, an adaptive multi-policy cache framework for CXL-enable DM systems with multiple *CNs*. Reinforcement Learning (RL) agent of *Alchemy* uses source-aware state vectors and multi-level actions to observe and manage the DRAM cache. It assigns rewards based on previous access events and the current action, which provides multi-host adaptivity. Additionally, *Alchemy* employs a lightweight Q-table lookup pipeline for efficient decision-making, which achieves 44x speedup than the state-of-the-art learning-based cache prototype.

[†] These authors are contributed equally to this work.

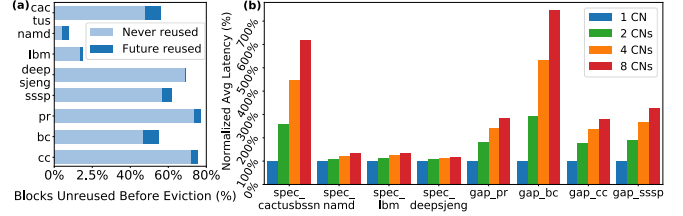


Fig. 1. Dynamic characteristics of workloads running on multiple *CN*. (a) The percentage of unreused blocks before eviction in 4-*CN* platform, including the blocks reused in the future or not. (b) The latency slowdown in the scenario of multi-*CN* contention. Latency was normalized to 1-*CN*-only cases.

II. REWARD-DRIVEN CACHE AGENT

In this section, we present the design of the RL agent for the reward-driven cache policy of *Alchemy*. The primary goal of the cache agent is to improve the overall response latency of *MN* and adapt to the changing environments with dynamic workloads in DM systems.

As shown in Fig. 2(a), *Alchemy*’s cache agent consists of two parallel tasks: cache management decision-making and sampling-based reward feedback to the Q-table. At each time step t , the agent observes an page access event and generates the synthesized state vector S_t , then it selects the cache operation with the maximum value in the line S_t of the Q-table (1). At the same time, the agent collects specific state-action pairs and cache access events. Sampled records are pushed into the ring-buffer-like evaluation queues (EQs) (2). During this time, items dequeued from EQs are used for updating the Q-table via the on-policy SARSA algorithm (3) as follows:

$$Q[S_t, A_t] \leftarrow Q[S_t, A_t] + \alpha(R_{t+1} + \gamma Q[S_{t+1}, A_{t+1}] - Q[S_t, A_t]).$$

In this assignment, α denotes the learning rate for Q-value updates, and γ denotes the discount factor.

A. State-Action Pair for Multi-policy Co-optimization

Alchemy models the RL environment with a state vector ($SIGPN_t, SIGPC_t$) and action space $\{AR_0, AR_1, AR_2, ABP\}$, enabling adaptive decision-making.

State: For each memory access, the agent can extract the program features to select the most suitable action. In our study, the state S_t of *Alchemy*’s agent is defined as a vector with only two elements combining data access and control flow features ($SIGPN_t, SIGPC_t$).

Data accesses from different *CNs* may exhibit distinct patterns. To characterize the source-aware feature of data access, the page signature $SIGPN_t$ is hashed from the source port ID of the CXL PBR request flit and the page number (PN) of the current request. The cache agent can roughly distinguish accesses from different *CNs* to shared data pages and exploit implicit multi-*CN* access behaviors. As an additional hint for exploiting program behavior, the program counter (PC) signature $SIGPC_t$ captures the control flow of workloads running on each *CN* by combining the folded PC, computed using the H3 hash [4] function, with the DRAM cache hit/miss bit.

Action: In Task D, *Alchemy* supports two types of action for DRAM cache management with multiple policies: **bypassing and cache replacement with a hint**. When the memory request *hit* in the DRAM cache, the agent updates the reuse level counter of the requested block to level $\{0, 1, 2\}$, guided by a set of 3 actions $\{AR_0, AR_1, AR_2\}$. In a cache *miss*, the agent can choose the block

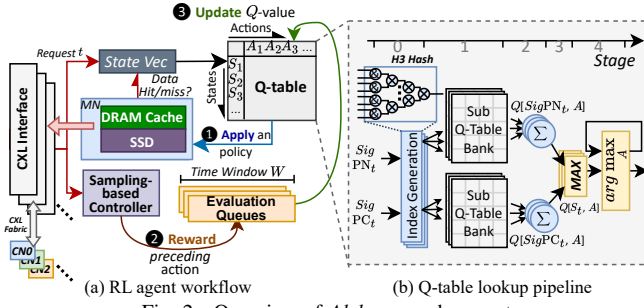


Fig. 2. Overview of *Alchemy* cache agent.

accessed from the SSD bypassed (action A_{BP}) or placed in the cache with new reuse level. For a replacement, the cache evicts the block with the lowest reuse level and assigns a reuse level value according to the Q-value-maximized action in the S_t entry of the Q-table.

B. Reward-Driven Adaptation

Alchemy is a reward-driven framework, and its adaptive optimization on MN performance is driven by the reward feedback R_{t+1} from the action applied at moment t . We define two reward functions $\{R_T, R_P\}$ to reflect the benefits from each action.

Firstly, R_T instantly rewards an action whose state-action pair remains in an EQ, triggered by the cache hit/miss events of the corresponding pages. When a request hits the DRAM cache page which is tracked by a EQ, the agent gives a positive reward R_T^H to the newest unrewarded state-action pair in the EQ. Otherwise, it provides a negative reward R_T^M . Secondly, R_P is a post-reward function calculated for evaluated actions (dequeued from EQs), forcibly assigning rewards to actions left unrewarded by R_T . Positive reward R_P^{CA} incentivize cache-averse actions, such as bypassing after a cache miss or updating the reuse level to R_0 after a cache hit, for non-reused pages during a lengthy evaluation process. Negative rewards R_P^{CF} are assigned for other cache-friendly actions.

C. Tile-coded Q-table lookup pipeline

To further minimize the overhead on real-world hardware, we design a tile-coded Q-table lookup pipeline, as shown in Fig. 2(b). It can accurately resolve the state feature with comparable generalizability [5] and lighter overhead than a monolithic Q-table. We split the Q-table into multiple table groups for each feature in the state vector, with each group containing small sub-tables for storing partial Q-values of feature-action pairs. First, H3 hash functions [4] generate multiple indexes for each feature with different seed sets $\{\{\pi_i\}_k\}$. The i -th bit of index I can be calculated as: $I(i) = \bigotimes_{j=0}^n x(j) \wedge \pi_j(i)$. $\bigotimes$ is bit-level eXclusive OR and $\wedge$ is bit-level AND operation. Then, we use these indexes to look up and obtain partial Q-values of all actions for each state feature from the sub-tables, and an array of adder trees merges them into complete Q-values. Finally, the decision is driven by the feature-action pair with the maximum Q-value.

III. EVALUATION

We prototype the *Alchemy* with Xilinx Alveo U280 FPGA. In our system, the DRAM cache is configured with 64MB size, 4KB block size, and 8-way set associativity. The access characteristics of the CXL-attached SSD are migrated from [1]. The evaluation uses a series of workloads and an open-source memory tracing tool [1] is used to collect a series of memory access traces for evaluation. LRU serves as the baseline with two scenarios: partitioned cache and shared cache. In the partitioned scenario, static index-based partitioning allocates exclusive cache portions to each CN.

Alchemy's agent relies on broad state-action history, but limited hardware resources constrain it. To observe access events over a larger period with small on-chip memory, we propose a two-stage sampling strategy, shown in the sampling-based controller in Fig. 2(a). We

TABLE I
RESOURCE UTILIZATION AND DECISION-MAKING LATENCY

Module	BRAM	DSP	LUT	FF	Latency
LSTM [2]	339	145	85029	103561	46.3 ms
ICGMM [3]	8	113	58353	152583	3 μ s
Alchemy(Ours)	72	37	25218	19248	68 ns

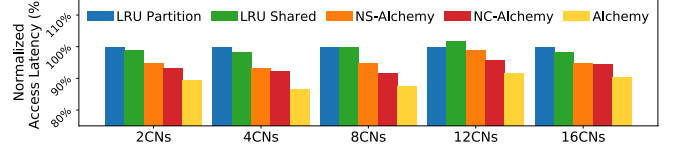


Fig. 3. Average latency reduction across different CN Numbers.

sample cache set groups where the lowest bits of set number match specific random offsets. Each group has a corresponding EQ for requests within the sampled sets. After accessing a sampled DRAM cache set, we enqueue its state-action pair only if the requested cacheline matches one of the specific sampled locations.

To evaluate the real-time performance and resource utilization of *Alchemy*, we compare it with the LSTM-based [2] and ICGMM [3] strategies. Under the current design parameters, the resource utilization is shown in Table I, running at 300 MHz on the FPGA. We pipeline the Q-Table lookup, achieving decision-making that is 44x faster than the hardware-efficient ICGMM method with only 68 ns of latency and comparable resource overhead.

We evaluated the cache management strategy of *Alchemy* on a multi-CN system, as shown in Fig. 3, the results are normalized with partitioned LRU as the baseline. As the number of CN increases, inter-CN interference becomes the primary bottleneck for memory performance. *Alchemy* can effectively identify critical data, evict less important data from the cache, or employ a bypass strategy to reduce cache pollution, resulting in a more efficient cache management strategy than all partitioned or shared LRU cache. Overall, it allows *Alchemy* to reduce average memory access latency from 8.71% to 14.56% compared to the fastest LRU baseline.

We also evaluated the impact without request source and control flow tracking in a multi-CN system. As shown in Fig. 3, NS-*Alchemy* refers to *Alchemy* without hashed source port IDs, and NC-*Alchemy* represents *Alchemy* without PC information. Including only control flow information reduces the average access latency by only 4.67%, while including only source information reduces it by 6.78%. These results indicate that source tracking and control flow jointly optimization can guide more precise cache management.

ACKNOWLEDGMENT

This work was supported in part by the Strategic Priority Research Program of the Chinese Academy of Sciences, Grant No. XDB0660101, XDB0660000, and XDB0660100, in part by the National Natural Science Foundation of China under Grant and 62172380, in part by Jiangsu Provincial Natural Science Foundation under Grant BK20241818, in part by Youth Innovation Promotion Association CAS under Grant Y2021121, in part by USTC Research Funds of the Double First-Class Initiative under Grant YD2150002011.

REFERENCES

- [1] S.-P. Yang and et.al, "Overcoming the Memory Wall with {CXL-Enabled} {SSDs}," in *ATC*, 2023.
- [2] Z. Shi and et al., "Applying deep learning to the cache replacement problem," in *MICRO*, 2019.
- [3] H. Chen and et al., "ICGMM: CXL-enabled memory expansion with intelligent caching using gaussian mixture model," in *DAC*, 2024.
- [4] Ramakrishna and et al., "Efficient hardware hashing functions for high performance computers," *IEEE Transactions on Computers*, 1997.
- [5] R. Bera and et al., "Pythia: A Customizable Hardware Prefetching Framework Using Online Reinforcement Learning," in *MICRO*, 2021.